Lập trình mạng · Chương 2

Bài toán ứng dụng: server tra cứu phục vụ nhiều client

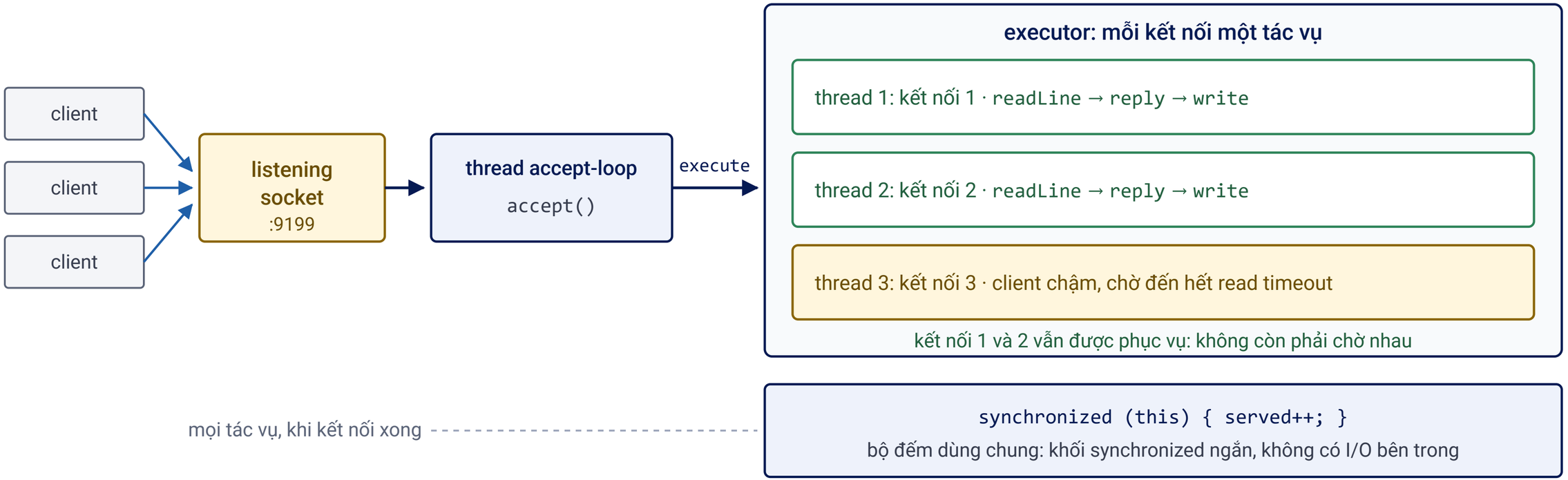
TS. Đặng Ngọc Hùng

Server tra cứu cần phục vụ song song và dừng được có trật tự

1. Server tuần tự: client đến sau một client chậm phải chờ **gần 4 giây**
2. Đã có đủ công cụ: thread, đồng bộ hóa, **executor**, **virtual thread**
3. Yêu cầu mới: **dừng server có trật tự** khi vẫn còn kết nối đang phục vụ

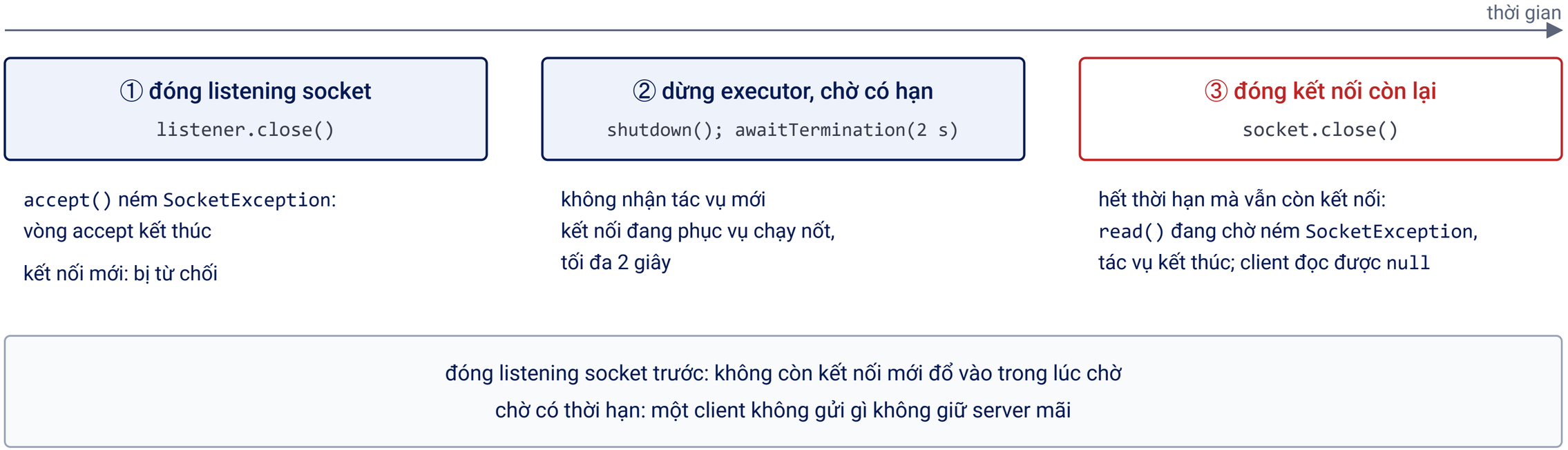
> **Câu hỏi của phần:** ghép executor vào server tra cứu thế nào, và dừng server ra sao khi vẫn còn kết nối?

Vòng accept chỉ nhận kết nối, mỗi kết nối là một tác vụ của executor



Thread-per-connection: mỗi kết nối một tác vụ, chạy trên một thread từ đầu đến cuối; dữ liệu dùng chung chỉ đụng tới trong khối `synchronized` ngắn.

Dừng server có trật tự theo ba bước



Graceful shutdown: ngừng nhận kết nối mới, chờ có thời hạn các kết nối đang phục vụ, rồi đóng phần còn lại.

Client chậm không còn chặn client khác

Java

```
void serve() {
    while (!stopping) {
        try {
            Socket connected = listener.accept();
            workers.execute(() -> serveConnection(connected));  // thread-per-connection: mỗi kết nối một tác vụ
        } catch (IOException e) {  // đóng listening socket để dừng server cũng làm accept() ném SocketException
            log.accept((stopping ? "stopping, accept loop ends: " : "accept error: ") + e.getMessage());
        }
    }
}
```

Kết quả chạy

```
     4 ms  listening on port 9199 with a pool of 8 threads
  1599 ms  accepted remote port 57266
  2633 ms  accepted remote port 57267
  2660 ms  closed remote port 57267, requests: 1
  6616 ms  connection error, remote port 57266: Read timed out
```

200 kết nối giữ 1 giây: pool 8 thread mất 25 giây, virtual thread mất hơn 1 giây

```java
ExecutorService workers = virtual
        ? Executors.newVirtualThreadPerTaskExecutor()        // mỗi kết nối một virtual thread
        : new ThreadPoolExecutor(8, 8, 0, TimeUnit.SECONDS,  // 8 platform thread
                new ArrayBlockingQueue<>(200), new ThreadPoolExecutor.CallerRunsPolicy());
```

Kết quả chạy

```
pool: 200 connections, 200 OK replies, slowest reply after 24388 ms, all done in 25497 ms
virtual: 200 connections, 200 OK replies, slowest reply after 202 ms, all done in 1238 ms
```

stop() chạy đủ ba bước khi vẫn còn một kết nối mở

Java

```
stopping = true;
listener.close();  // 1. ngừng nhận kết nối mới: accept() đang chờ ném SocketException
log.accept("stop 1/3: listening socket closed");
workers.shutdown();  // 2. không nhận tác vụ mới; chờ các kết nối đang phục vụ, có thời hạn
log.accept("stop 2/3: waiting up to " + graceMillis + " ms for " + open.size() + " open connection(s)");
if (!workers.awaitTermination(graceMillis, TimeUnit.MILLISECONDS)) {
    log.accept("stop 3/3: closing " + open.size() + " remaining connection(s)");
    for (Socket socket : open) {
        socket.close();  // 3. đóng kết nối còn lại: read() đang chờ ném SocketException
    }
}
```

Kết quả chạy

```
   69 ms  server: accepted remote port 60394
   92 ms  client: hanoi -> OK Thủ đô của Việt Nam
 1097 ms  server: stop 1/3: listening socket closed
 1097 ms  server: stopping, accept loop ends: Socket closed
 1109 ms  server: stop 2/3: waiting up to 2000 ms for 1 open connection(s)
 3110 ms  server: stop 3/3: closing 1 remaining connection(s)
 3114 ms  client: readLine() -> null (server closed the connection)
 3115 ms  server: connection error, remote port 60394: Socket closed
 3115 ms  server: stopped, connections served: 1
```

Vòng accept phải phân biệt lỗi do dừng server với lỗi thật

1. `listener.close()` làm `accept()` đang chờ ném `SocketException: Socket closed`, giống một lỗi
2. Vòng accept phân biệt bằng cờ `stopping`: đang dừng thì **thoát vòng lặp**; lỗi thật thì **ghi log và nhận tiếp**
3. Không phân biệt: server báo lỗi giả mỗi lần dừng, hoặc dừng hẳn vì một lỗi tạm thời

Câu hỏi 1

Khi dừng server tra cứu đồng thời, vì sao bước đầu tiên là đóng listening socket?

A. Để ngừng nhận kết nối mới, trong khi các kết nối đang phục vụ vẫn chạy nốt.

B. Để đóng luôn mọi kết nối đang phục vụ.

C. Vì executor chỉ dừng được khi listening socket đã đóng.

D. Để các client đang kết nối nhận được connection refused.

Câu hỏi 1

Khi dừng server tra cứu đồng thời, vì sao bước đầu tiên là đóng listening socket?

A. Để ngừng nhận kết nối mới, trong khi các kết nối đang phục vụ vẫn chạy nốt.

B. Để đóng luôn mọi kết nối đang phục vụ.

C. Vì executor chỉ dừng được khi listening socket đã đóng.

D. Để các client đang kết nối nhận được connection refused.

Listening socket và các connected socket là những socket khác nhau: đóng listening socket chỉ ngừng nhận kết nối mới, kết nối đang phục vụ không bị ảnh hưởng và được chờ ở bước hai. Connection refused chỉ xảy ra với kết nối mới.

Câu hỏi 2

Server tra cứu chuyển sang virtual thread. Server có còn cần read timeout cho mỗi kết nối không?

A. Không, vì virtual thread nhả carrier khi chờ nên chờ bao lâu cũng không tốn gì.

B. Không, vì virtual thread tự kết thúc sau một thời gian.

C. Có, vì kết nối chờ mãi vẫn giữ socket và bộ nhớ của tác vụ; client không gửi gì có thể giữ tài nguyên không giới hạn.

D. Có, nhưng chỉ khi số kết nối vượt quá 10 000.

Câu hỏi 2

Server tra cứu chuyển sang virtual thread. Server có còn cần read timeout cho mỗi kết nối không?

A. Không, vì virtual thread nhả carrier khi chờ nên chờ bao lâu cũng không tốn gì.

B. Không, vì virtual thread tự kết thúc sau một thời gian.

C. Có, vì kết nối chờ mãi vẫn giữ socket và bộ nhớ của tác vụ; client không gửi gì có thể giữ tài nguyên không giới hạn.

D. Có, nhưng chỉ khi số kết nối vượt quá 10 000.

Virtual thread làm việc chờ rẻ hơn, nhưng mỗi kết nối vẫn giữ một socket của hệ điều hành và bộ nhớ của tác vụ. Không có read timeout, client không gửi gì giữ các tài nguyên đó mãi.

Tổng kết

1. **Thread-per-connection**: vòng accept giao mỗi kết nối cho executor; client chậm chỉ giữ thread của nó

2. Kết nối chủ yếu chờ I/O: **virtual thread** phục vụ 200 kết nối cùng lúc; read timeout vẫn cần

3. **Graceful shutdown**: đóng listening socket → dừng executor, chờ có hạn → đóng kết nối còn lại

Từ khóa cần nhớ thread-per-connection | graceful shutdown | ExecutorService | virtual thread

Mở rộng

Chủ đề nâng cao

- Giới hạn số kết nối đồng thời bằng Semaphore
- Đo thông lượng bằng công cụ tạo tải

Bài tập mở rộng

Thêm giới hạn 100 kết nối đồng thời: kết nối thứ 101 nhận phản hồi BUSY rồi bị đóng.

Tổng kết chương 2

1. **TCP**: listening socket chờ, connected socket trao đổi; bắt tay ba bước; timeout, half-close, phân loại lỗi
2. **Bài toán ứng dụng đầu tiên**: server tra cứu tuần tự đúng đặc tả; 4-tuple; integration test với port 0
3. **Đa luồng**: thread và race condition; synchronized, happens-before, wait/notify; executor, virtual thread
4. **Bài toán ứng dụng thứ hai**: server tra cứu đồng thời, thread-per-connection, graceful shutdown

Tiếp theo

UDP: datagram và DatagramSocket